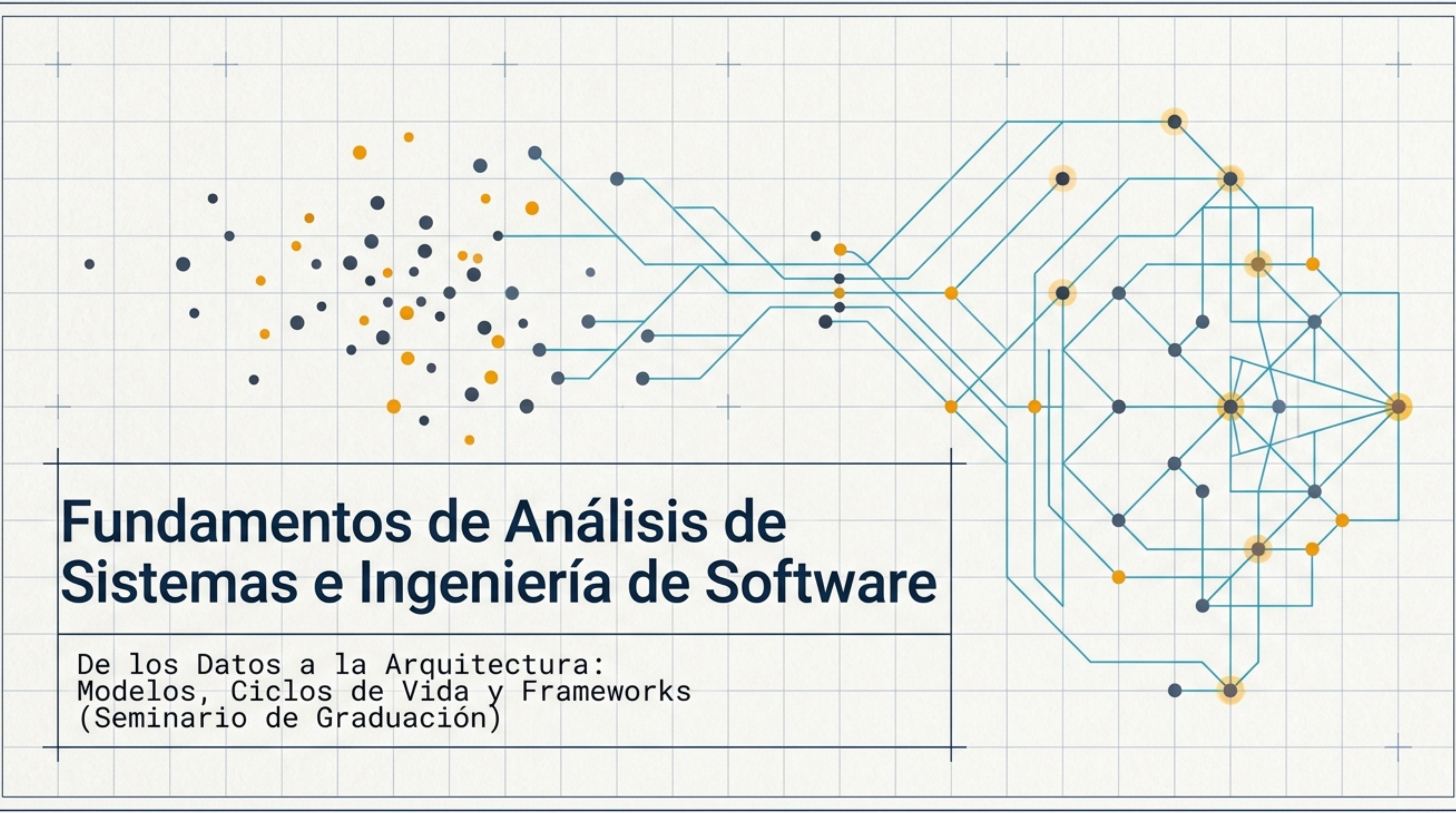
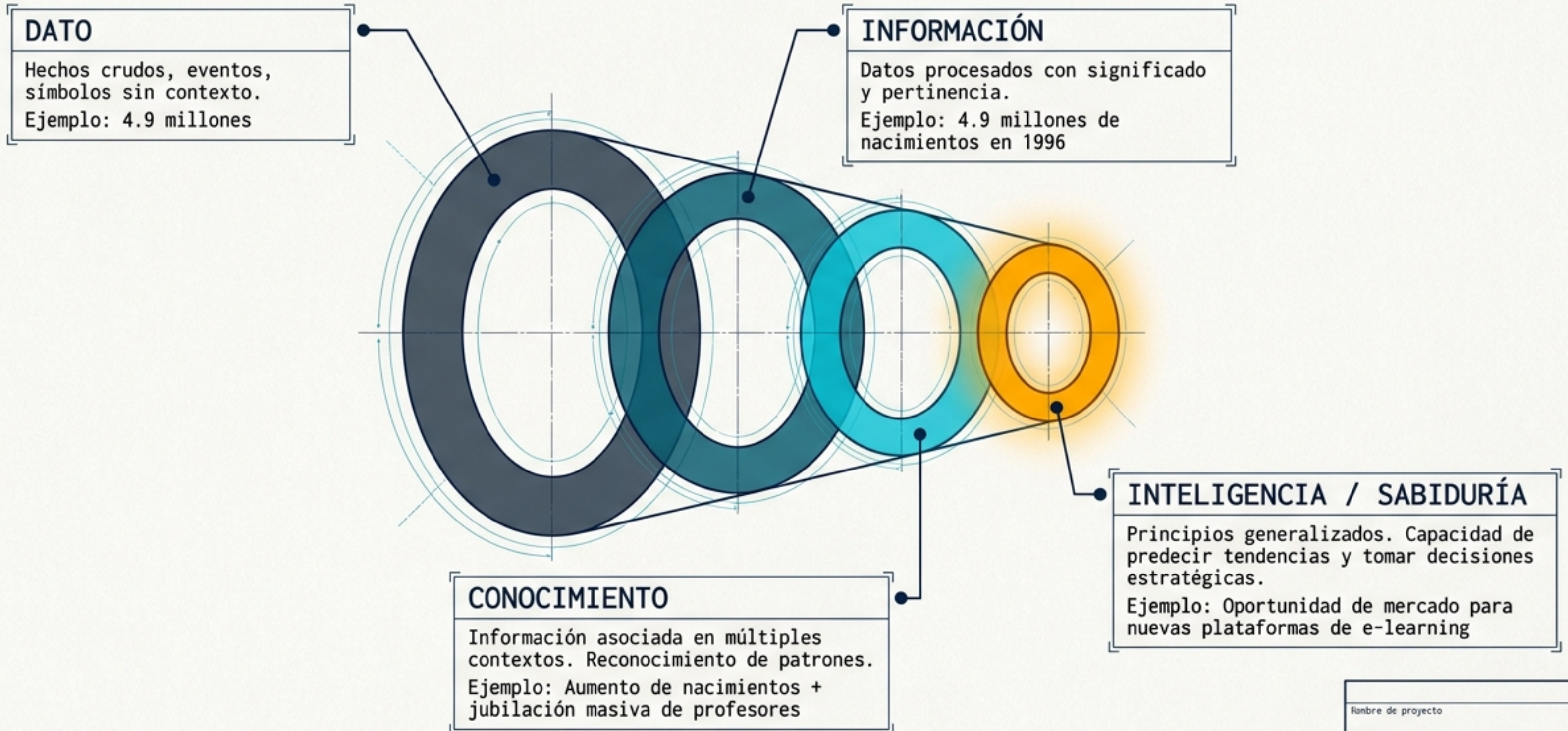




Fundamentos de Análisis de Sistemas e Ingeniería de Software

De los Datos a la Arquitectura:
Modelos, Ciclos de Vida y Frameworks
(Seminario de Graduación)

El código procesa datos, pero los sistemas de información deben generar inteligencia.



Nombre de proyecto		
Fecha	Rev.	Rev.

ANATOMÍA DE UN UN SISTEMA

Entidades interrelacionadas con un fin común.

PROCESAMIENTO DE VALOR

Un Sistema de Información (SI) transforma datos en información oportuna, relevante y confiable.

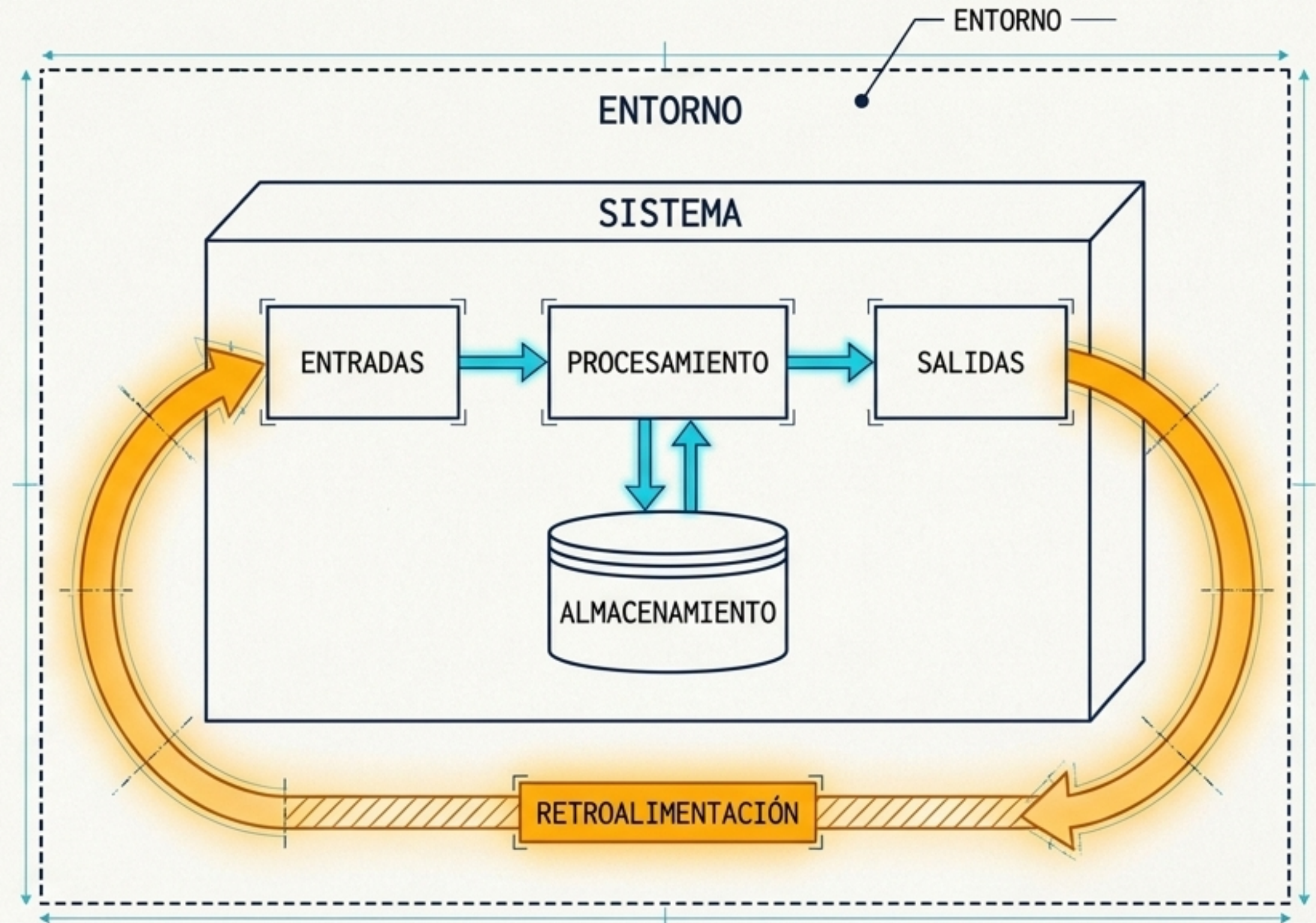
COMPONENTES VITALES

Captura (Entradas) -> Transformación (Proceso) -> Persistencia (Almacenamiento) -> Reportes (Salidas).

INTERACCIÓN CON EL ENTORNO

Sistemas Cerrados: Aislados (teóricos).

Sistemas Abiertos: El software real interactúa y se adapta constantemente a variables externas mediante la retroalimentación.



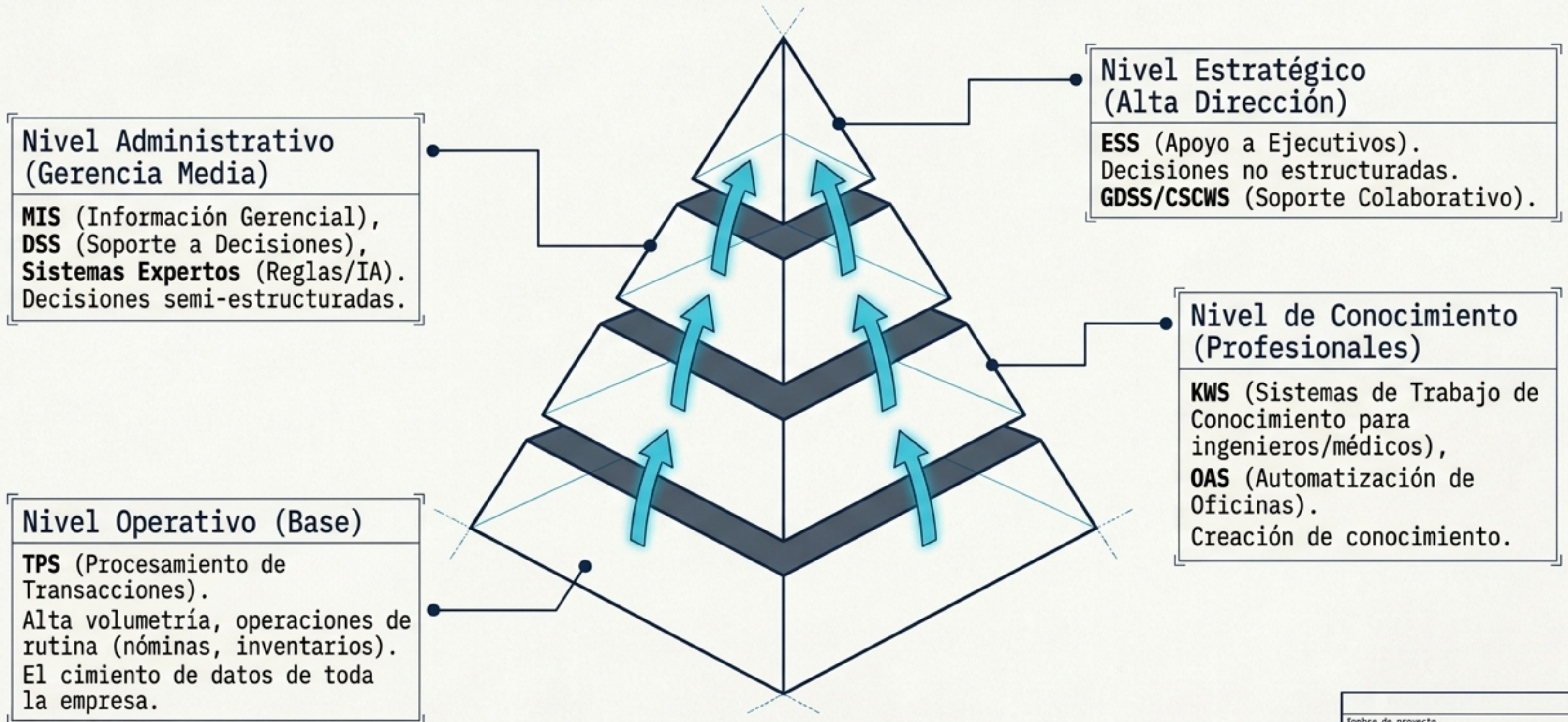
Nombre de proyecto

Fecha

Rev.

Rev.

La arquitectura del software depende de su posición en la pirámide organizacional.



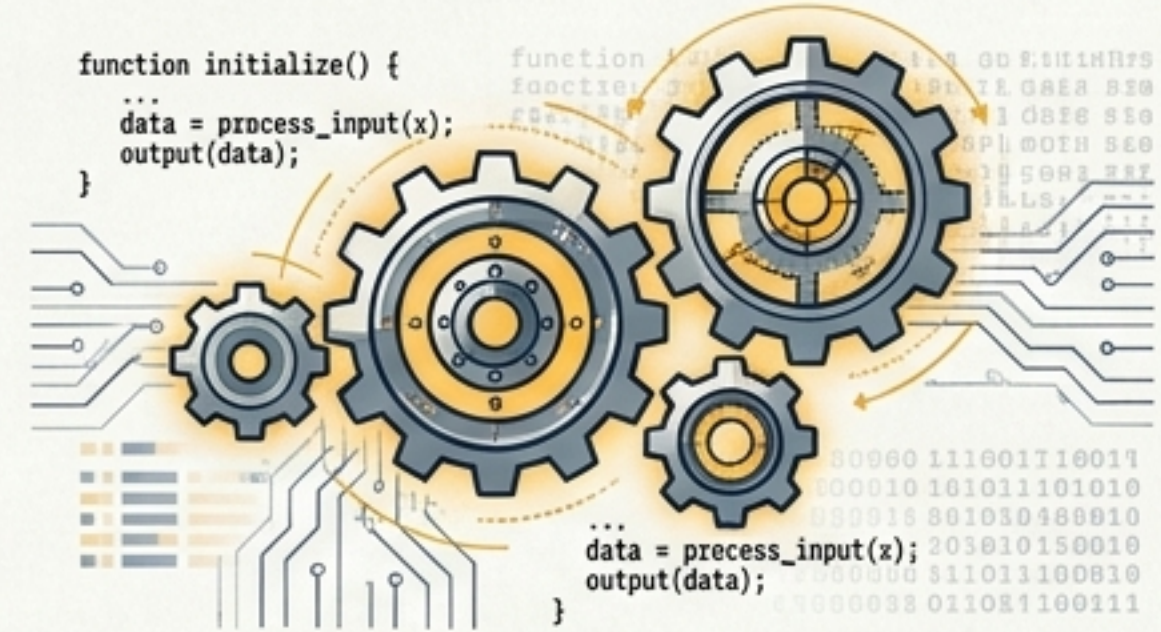
Nombre de proyecto		
Fecha	Rev.	Rev.

El Análisis define el problema; la Ingeniería construye la solución.



Análisis de Sistemas (El Qué y Por Qué)

- Traduce problemas del mundo real a requerimientos técnicos.
- Analiza reglas de negocio y necesidades de los usuarios.
- Diseña procedimientos y flujos lógicos.



Ingeniería de Software (El Cómo)

- Aplica principios de ingeniería cuantificables y sistemáticos.
- Diseña la arquitectura técnica (n-capas, BD).
- Construye, opera y mantiene software confiable y económico.

Sistemas de Información

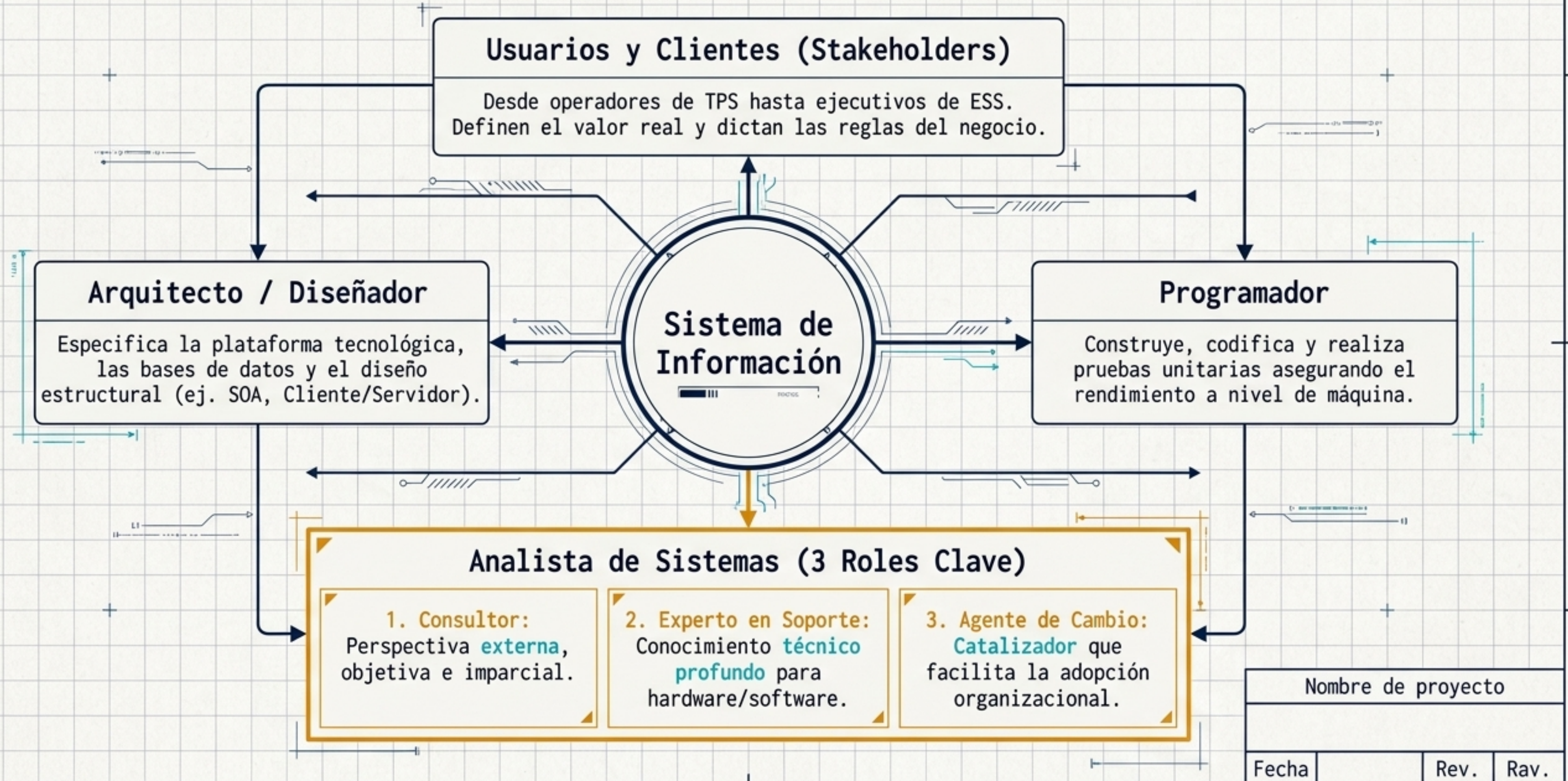
El **Análisis** entrega el diseño lógico y los requerimientos



La **Ingeniería** dicta la arquitectura, el código y el despliegue.

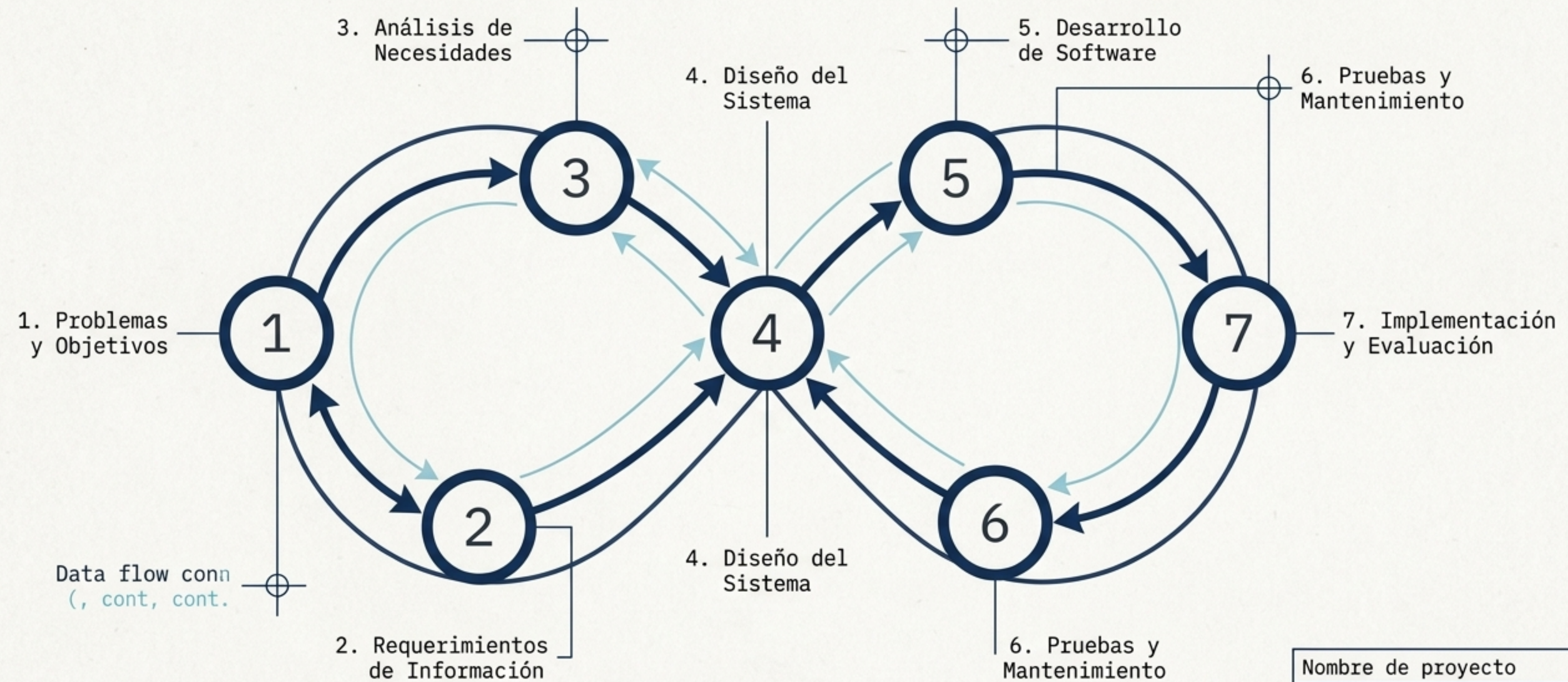
Nombre de proyecto		
Techa	Rev.	Rev.

El ecosistema humano es tan crítico como la plataforma tecnológica.



El Ciclo de Vida del Desarrollo de Sistemas (SDLC): Un mapa estructurado y metódico.

El SDLC garantiza que los sistemas resuelvan problemas reales mediante fases estructuradas. Aunque es secuencial, exige iteración constante.



Nombre de proyecto

Fecha Rev. Rav.

Fases 1 a 3: De la observación del problema a la especificación lógica.

1



Identificación (Problemas y Objetivos)

- Observación objetiva del entorno.
- ¿Qué intenta lograr la empresa?

Entregable: Informe de viabilidad (técnica, económica, operativa). Dicta si el proyecto vive o muere.

2

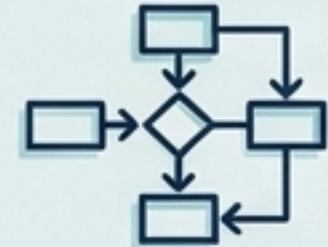


Requerimientos Humanos (Información)

- Enfoque altamente interactivo y psicológico.
- Herramientas: Entrevistas, cuestionarios, observación (STROBE), muestreo.

Objetivo: Entender el quién, qué, dónde, cuándo y cómo.

3



Análisis de Necesidades (Sistemas)

- Traducción a herramientas lógicas.
- Uso de Diagramas de Flujo de Datos (DFDs), Diccionarios de Datos y Casos de Uso.

Entregable: Propuesta de Sistema recomendada.

Nombre de proyecto

Fecha

Rev.

Rav.

Fases 4 a 7: De la arquitectura técnica a la adopción y mantenimiento continuo.

4. Diseño del Sistema (Lógico/Físico)

- Diseño de interfaces de usuario (HCI) y controles de entrada.
- Especificaciones de bases de datos y arquitectura.

5. Desarrollo de Software

- Programación activa, creación de manuales y manuales y ayudas en línea.
- El puente donde el Analista entrega planos planos al Programador.

6. Pruebas y Mantenimiento

- Verificación con datos reales y de prueba.
- **Dato Crítico:** El mantenimiento es la fase más larga y costosa (hasta 70% del presupuesto histórico de TI).

7. Implementación y Evaluación

- Capacitación, conversión de datos y paso a producción.
- Evaluación final de utilidad en dasta es en tiempo, forma y lugar.

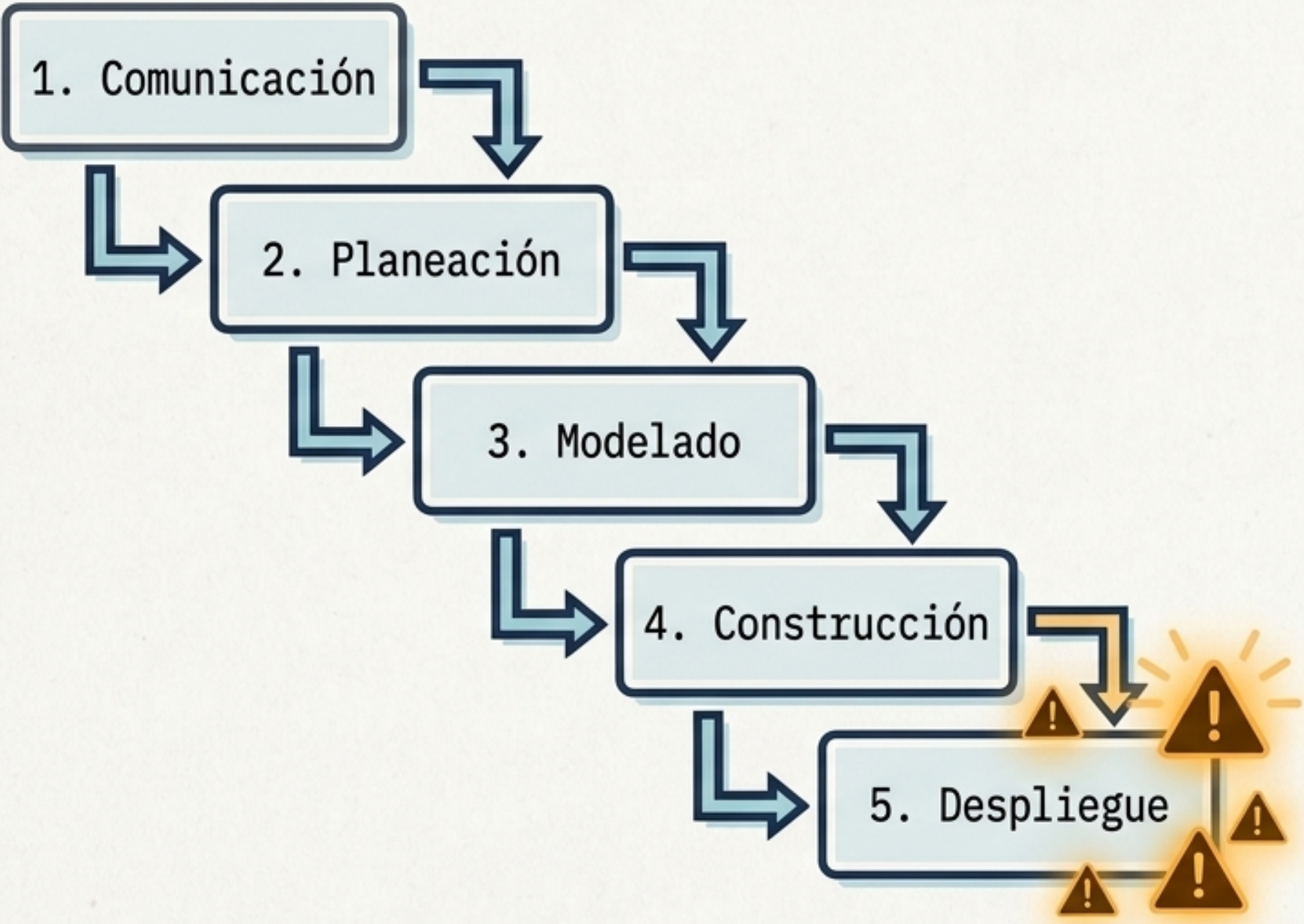
Nombre de proyecto

Fecha

Rev.

Rav.

El Modelo en Cascada: Rigor prescriptivo con alta vulnerabilidad al cambio.



Características Clave

- Paradigma prescriptivo clásico; fases estrictamente secuenciales.
- Exige requerimientos 100% congelados desde el día uno.
- Documentación exhaustiva en cada transición.

Riesgos y Evolución

- El cliente no ve software funcional hasta el final del ciclo.
- La regla del costo: Los cambios tardíos son **financieramente catastróficos** y destruyen la arquitectura.
- Ideal solo para: **Sistemas de misión crítica** con variables inmutables.

Nombre de proyecto		
Fecha	Rev.	Rav.

Modelo Incremental: Mitigación de riesgos mediante entregas de valor temprano.

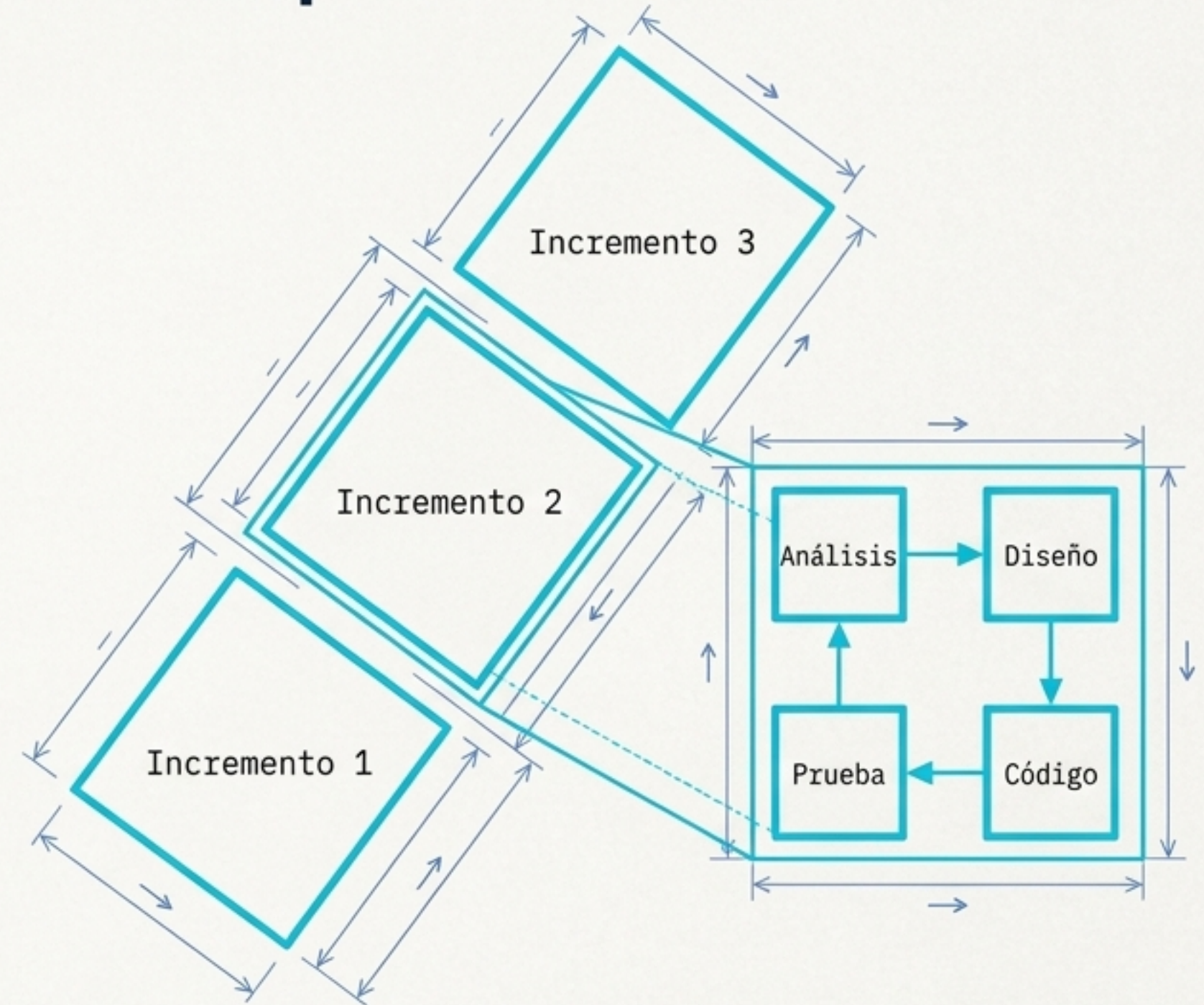
El Principio Iterativo

El aprendizaje humano es iterativo; la ingeniería de software también debe serlo.

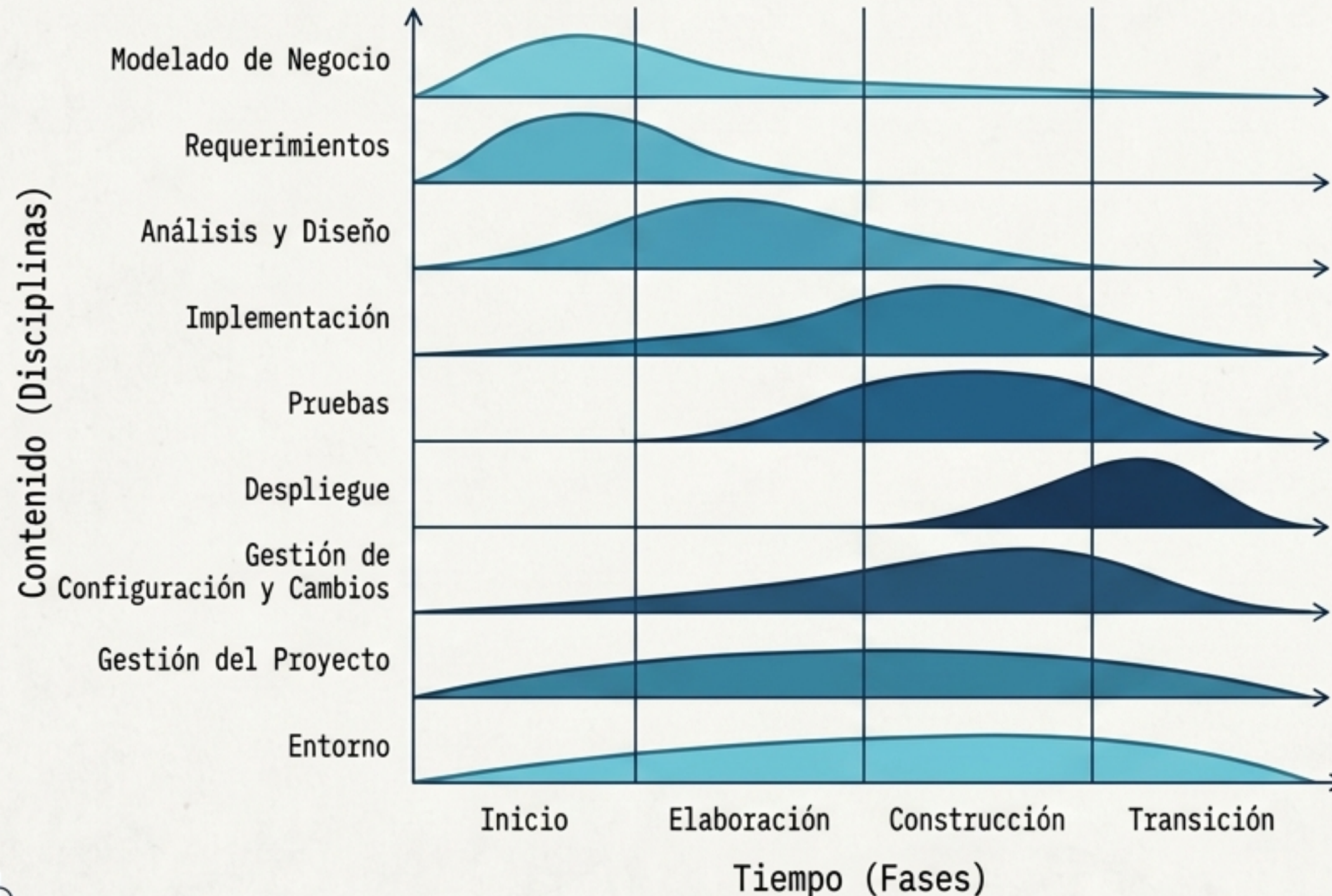
Entrega el software en partes funcionales, atacando primero la funcionalidad de mayor riesgo.

Características y Ventajas

- **Regla del 80%:** Ingeniería suficiente para que el incremento funcione; el refinamiento ocurre después.
- **Prototipos Operacionales:** El cliente obtiene valor de negocio rápidamente, no solo documentos.
- Reducción drástica de riesgos arquitectónicos y máxima flexibilidad ante requerimientos volátiles.



RUP (Rational Unified Process): El framework unificado y centrado en la arquitectura.



Definición:

Un proceso de desarrollo creado por Rational (IBM) que es iterativo, incremental, guiado por Casos de Uso y estructurado a través de modelado visual (UML).

Las 6 Mejores Prácticas de RUP

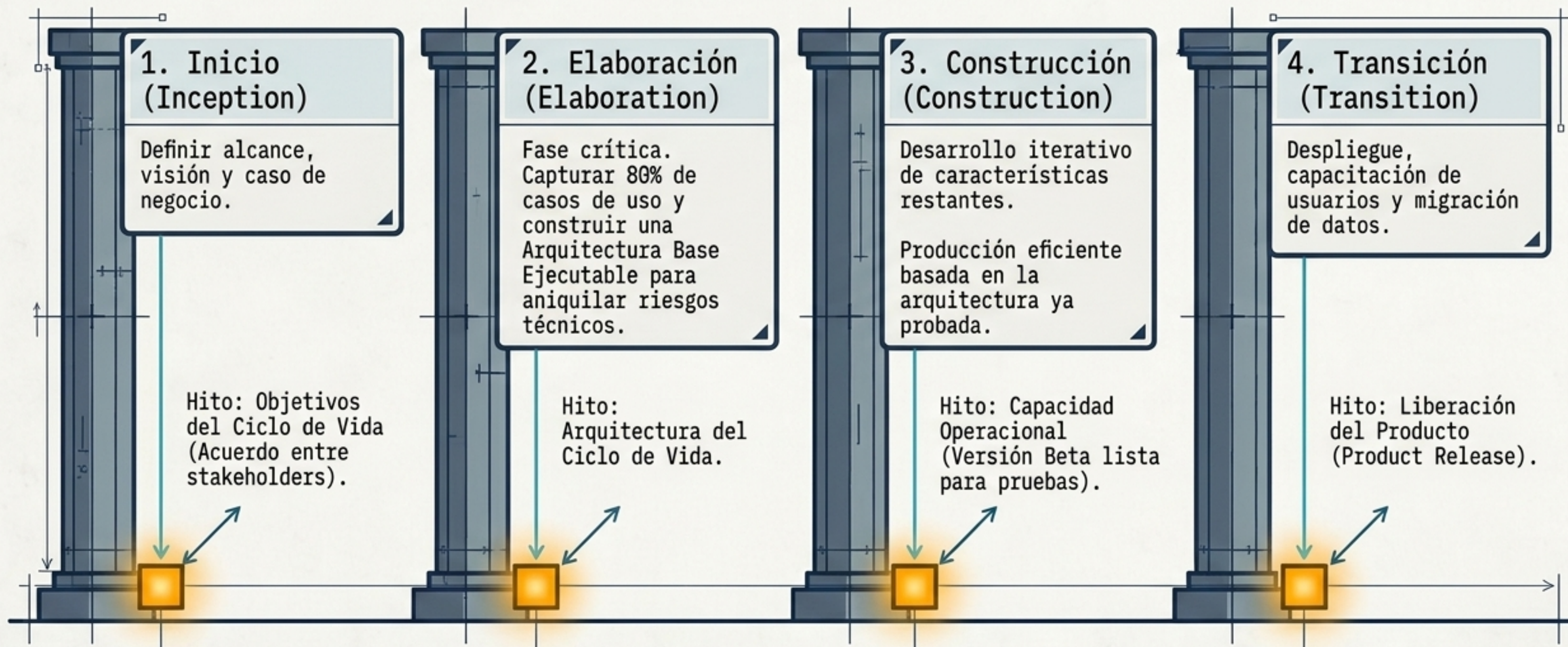
1. Desarrollo iterativo de software.
2. Gestión rigurosa de requerimientos.
3. Arquitectura basada en componentes reutilizables.
4. Modelado visual estandarizado (UML).
5. Verificación continua de la calidad.
6. Control y gestión de cambios.

Nombre de proyecto

Fecha / Rev.

Revisión /

Las 4 fases temporales de RUP: Construyendo hacia hitos específicos.



Nombre de proyecto

Data 05 / 2023

Revisión /

Diagnóstico Arquitectónico: ¿Cuándo utilizar SDLC frente a RUP?

Dimensión	SDLC (Kendall)	RUP (Rational)
Enfoque Principal	Sistemático y centrado en procesos de negocio.	Iterativo y centrado en mitigación de riesgos técnicos.
Estructura del Ciclo	7 Fases funcionales (a menudo aplicadas en cascada o iteración lenta).	4 Fases temporales transversales a múltiples disciplinas.
Modelado Base	Análisis Estructurado (DFDs, Diccionarios de datos).	Orientado a Objetos (Casos de Uso, UML).
Gestión del Riesgo	Evaluated en la fase de viabilidad inicial.	Mitigado mediante la creación temprana de una arquitectura ejecutable.
Artefactos Centrales	Propuesta de sistemas, manuales de flujo.	Documento de visión, Modelos UML, Prototipos.

Principios inmutables para el Arquitecto de Sistemas moderno

El Software es Capital

No escribimos líneas de código; construimos arquitecturas de información que generan inteligencia y resuelven problemas de negocio.

La Calidad Nace en el Diseño

La calidad no se inyecta en la etapa de pruebas; nace de un modelado profundo (Análisis) y una arquitectura ejecutable (Elaboración).

El Ingeniero Moderno

Dominio Metodológico

Usa SDLC para auditoría y estructuración de procesos, Incremental para adaptabilidad, y RUP para sistemas complejos orientados a objetos.

El Factor Humano

La maestría técnica es inútil sin análisis organizacional. Un software perfecto fracasará sin la aceptación del usuario y el alineamiento cultural.